國立臺南大學資訊工程學系

113 級畢業專題報告

增強式學習演算法於多智能體系統中的競爭與合作策略探究

Exploration of Competitive and Cooperative Strategies of Reinforcement Learning Algorithms in Multi-Agent Systems

專案編號：NUTN-CSIE-PRJ-113-009

組員：S10959009 林奕君

S10959022 陳馨柔

指導老師：朱明毅 博士

中華民國 113 年 1 月

國立臺南大學資訊工程學系
畢業專題報告審定書

增強式學習演算法於多智能體系統中的競爭與合作策略探究

Exploration of Competitive and Cooperative Strategies of Reinforcement Learning Algorithms in Multi-Agent Systems

學生　　學號: _____　姓名:_____

　　學號: _____　姓名:_____

所提之報告內容符合本學系大學部畢業專題實作標準。

指導教授：_____

系主任：_____

中華民國 113 年 1 月

增強式學習演算法於多智能體系統中的競爭與合作策略探究

摘要

隨著人工智慧的發展愈來愈成熟，許多機器學習演算法被提出且成功地解決了各種不同領域的問題。然而何種演算法適用在何種環境仍是許多使用者在意的議題，因此在本論文中我們探討了不同類型的增強式學習架構在多代理人系統處於不同環境下之效能。首先會先探討增強式學習的基本理論，了解增強式學習不同於其他機器學習技術的特性和特色。接著再探討由增強式學習架構所延伸出的演算法，包 Value-based 的 Hysteretic Q-Learning 和 CPM(Cooperative Policy Mechanism)與 Policy-based 的 WoLF PHC(Win or Learn Fast Policy Hill Climbing)共三種演算法。理解完各個演算法的核心概念後則會進行一種競爭型任務：Matching Pennies 和兩種合作型任務：Ball Balance 和 Pursuit Domain 的實驗，並根據實驗數據討論導致其結果的可能原因。最後再根據討論的結果做出結論。

關鍵詞: 增強式學習、Value-based 、Policy-based、合作、競爭

Exploration of Competitive and Cooperative Strategies of Reinforcement Learning Algorithms in Multi-Agent Systems

ABSTRACT

As the development of artificial intelligence becomes more and more mature, many machine learning algorithms have been proposed and successfully solved problems in various fields. However, which algorithm is suitable in which environment is still an issue that many users are concerned about. Therefore, in this paper, we explore the effectiveness of different types of reinforcement learning architectures in multi-agent systems in different environments. First, we discuss the basic theory of reinforcement learning, and then we explore the algorithms extended by the reinforcement learning architecture, including Value-based Hysteretic Q-Learning and CPM and Policy-based WoLF PHC. After understanding the core concepts of each algorithm, we conduct experiments on a competitive task: Matching Pennies and two cooperative tasks: Ball Balance and Pursuit Domain, and discuss the possible reasons for the results based on the experimental data. Finally, conclusions were drawn based on the discussion results.

Keywords: Reinforcement Learning、Value-based 、Policy-based、cooperative、competitive

致謝

我們能夠順利完成這次的專題，要感謝我們的指導老師朱明毅教授在整個專題研究期間給予我們許多指導和支持。在老師的教導下，我們不僅學習到許多關於增強式學習的理論知識，更獲得了豐富的實踐經驗，使我們獲益良多。老師總是樂於分享自己的想法及專業知識，使我們能夠更深入了解這個領域並在學術上有了更多的成長。

謝謝老師總是耐心解答我們的疑惑，悉心指導我們解決在研究中所遇到的困難，也願意花時間和我們一起討論很多問題，透過引導的方式帶著我們主動思考，讓我們在實作過程中培養了解決問題的能力。

除了學術上的指導外，老師也很願意分享在求學方面以及工作方面的自身經驗，提供了很多的啟示和引導，使我們在求學和工作的道路上更有信心和方向。

再次由衷感謝老師的細心指導，很榮幸能夠有老師作為我們的專題指導老師。在面對學業和事業上的挑戰，老師的諄諄教誨將成為我們堅持不懈的動力。我們也將帶著這份教導繼續成長，感謝老師對我們的無私奉獻。

林奕君 陳馨柔 謹致

國立臺南大學資訊工程學系

中華民國 113 年 1 月

目錄

摘要 i

ABSTRACT ii

致謝 iii

目錄 iv

圖目錄 vi

一、緒論 1

1.1 研究動機 1

1.2 論文架構 1

二、演算法介紹 2

2.1Reinforcement Learning 2

2.2Hysteretic Q-Learning 3

2.3WoLF Policy Hill-Climbing 4

2.4Cooperative Policy Mechanism 5

三、實驗環境與結果 6

3.1Ball Balance 6

3.1.1 實驗環境介紹 6

3.1.2 結果與討論 7

3.2Pursuit Domain 8

3.2.1 實驗環境介紹 8

3.2.2 結果與討論 9

3.3Matching Pennies 17

3.3.1 實驗環境介紹 17

3.3.2 結果與討論 17

四、結論 19

參考文獻 20

附錄 1 專題簡報 21

圖目錄

圖 1 增強式學習流程圖 2

圖 2 Hysteretic Q-Learning 演算法 3

圖 3 WoLF-PHC 演算法 4

圖 4 CPM 演算法 5

圖 5 平衡球任務示意圖 6

圖 6 球體位置比較圖 7

圖 7 球體速度比較圖 8

圖 8 網格環境示意圖 9

圖 9 成功捕捉一次所需最少步數-Hysteretic 10

圖 10 每回合平均捕捉成功次數-Hysteretic 10

圖 11 成功捕捉一次所需最少步數-WoLF 11

圖 12 每回合平均捕捉成功次數-WoLF 11

圖 13 成功捕捉一次所需的最少步數-CPM 12

圖 14 每回合平均捕捉成功次數-CPM 13

圖 15 成功捕捉一次所需最少步數-CPM(ε) 13

圖 16 每回合平均捕捉成功次數-CPM(ε) 14

圖 17 成功捕捉一次所需最少步數-CPM(λ) 14

圖 18 每回合平均捕捉成功次數-CPM(λ) 15

圖 19 成功捕捉一次所需最少步數 16

圖 20 每回合平均捕捉成功次數 16

圖 21 獎勵函數圖 17

圖 22 擲出正面比例圖-Hysteretic 18

圖 23 擲出正面比例圖-WoLF-PHC 18

一、 緒論

1.1 研究動機

隨著科技的蓬勃發展，人工智慧融入在我們的生活當中已成為趨勢。工業的自動化生產、倉儲業的物流搬運、服務業的送餐機器人等等，都是利用智慧機器人來協助工作，達到節省時間、產量最大化的效果。

然而由於產量最大化的要求，單一機器人已經無法滿足人們的需求，許多工作都是若只有單一個體是無法順利完成的。因此人們開始探索機器人之間的協作模式，透過各種演算法研究出機器人間的合作，透過互相協調、合作，共同完成一項任務。不僅能增加產量，也能減少時間成本。但是因多工合作會面臨到許多難以解決的問題，因此如今尚未找到最佳的機器學習方法。

因此本文將探討三種基於增強式學習架構的演算法並應用到一種競爭型任務、兩種合作型任務上，根據實驗結果討論原因並做出結論。

1.2 論文架構

本文會分成四個章節。第二章會探討增強式學習的基本觀念以及 Hysteretic Q-Learning、WoLF-PHC、Cooperative Policy Mechanism 三種演算法。第三章會講述三種實驗環境，包含合作型任務：Ball Balance、Pursuit Domain 以及競爭型任務：Matching Pennies 並比較各個演算法的實驗數據。第四章會對本文做出結論。

二、 演算法介紹

第二章將會介紹增強式學習和三種演算法,首先第 2.1 節會介紹增強式學習的理論;第 2.2 節中將介紹 Hysteretic Q-Learning 演算法;第 2.3 節中將介紹 WoLF Policy Hill-Climbing 演算法;第 2.4 節中將介紹 Cooperative Policy Mechanism 演算法。

2.1 Reinforcement Learning

增強式學習（Reinforcement Learning）是一種機器學習技術，可訓練智能體做出決策，以實現最佳結果。代理人在觀察環境後，採取一種行為，隨後環境反饋給代理人進行策略調整時所需之獎勵回報。代理人能透過不斷的自我探索學習以達到最終目標。以下是增強式學習的五個基本要素:

1. 代理人（agent）：自主學習的機器
2. 環境（environment）：代理人執行動作時的場所
3. 狀態（state）：代理人位於環境中的狀態
4. 動作（action）：代理人執行的動作
5. 獎勵（reward）：對於代理人的策略給出的獎勵評價

當代理人處於目前環境的一個狀態時，會根據自己學習的策略中選擇最適合的動作並執行，接著環境會反饋給代理人獎勵值並獲得下一個狀態，接著代理人再透過獎勵值調整策略。代理人進入到下一個狀態並進行下一回合的決策。圖 1 為增強式學習流程圖。

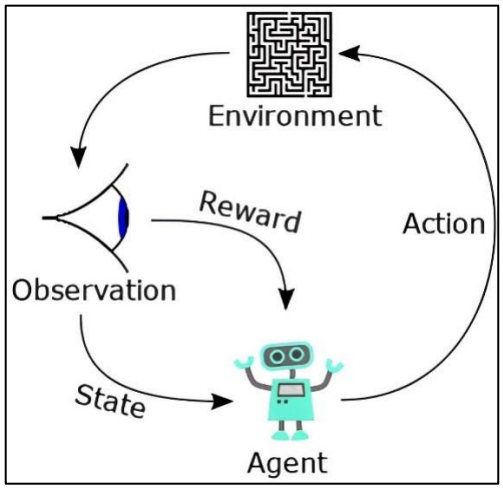


圖 1 增強式學習流程圖

2.2 Hysteretic Q-Learning

該算法對合作式多代理人系統(Multi-agent Systems, MAS)中協調學習的現有算法進行延伸，並擴展出了一種適用於獨立學習機器人(IL)，不需要機器人之間進行額外的通信的 Q-Learning 算法，稱為遲滯 Q-Learning。遲滯 Q-Learning 的收斂接近於集中式 Q-Learning。此外，計算得到的策略優於現有算法的結果。因此，遲滯 Q 學習是一種吸引人的用於合作式 MAS 中協調學習的去中心化強化學習算法。此算法在各種應用中，如雙矩陣博弈、合作式平衡球任務和追蹤領域等皆佔了優勢。

在 Hysteretic Q-Learning [1] 演算法中會使用到兩個不同的學習率α和β。若在(1)式計算出$\delta<0$ 時，代表受到不好的獎勵值懲罰，在(2)式則使用較小的學習率β更新 Q 值以減少懲罰。而$\delta \geq 0$ 則代表收到好獎勵值，此時則使用較大的學習率α更新 Q 值，讓代理人加速學習。此演算法可根據代理人的狀況給出相對應的學習率，透過這種方式能讓代理人加快學習速度以達到收斂。以下(1)式和(2)式為計算 Q 值的公式，其中 r 為獎勵，γ 為折扣率。圖 2 為 Hysteretic Q-Learning 演算法的完整流程圖。

$$\delta \leftarrow r + \gamma \max_{a'} Q_i(s',a') - Q_i(s,a_i) \tag{1}$$

$$Q_i(s,a_i) \leftarrow \begin{cases} Q_i(s,a_i) + \alpha\delta & \text{if } \delta \geq 0 \\ Q_i(s,a_i) + \beta\delta & \text{else} \end{cases} \tag{2}$$

```
設學習率α > β，初始化 Q_i(s,a_i) ← 0
重複執行(在所有回合中):
    初始化狀態 s
    重複執行(在回合的每一步中):
        (a)在狀態 s 以 Q 決策選擇出動作 a_i
        (b)執行動作 a_i，得到獎勵值 r 與下一狀態 s'
        (c)更新Q_i(s,a_i)
            δ ← r + γ max_a' Q_i(s',a') − Q_i(s,a_i)
            Q_i(s,a_i) ← { Q_i(s,a_i) + αδ   if δ ≥ 0
                         { Q_i(s,a_i) + βδ   else
        (d)更新狀態 s ← s'
```

圖 2 Hysteretic Q-Learning 演算法

2.3 WoLF Policy Hill-Climbing

Wolf Policy Hill-Climbing [2] 演算法的基本概念是以鼓勵收斂(Convergence)的方式改變算法使用的學習率，而不犧牲合理性(Rationality)。這個原理有一個簡單的直覺：輸的時候學得快，贏的時候學得慢。確定代理人何時獲勝的具體方法是將當前策略的預期收益與一段時間內平均策略的預期收益進行比較。這一原則有助於收斂，讓其他玩家有更多時間適應最初看起來有益的玩家策略變化，同時允許玩家更快地適應其他玩家有害的策略變化。圖 3 為 WoLF-PHC 演算法的流程。

在此演算法中，與 Hysteretic Q-Learning 演算法不同的部分在於它使用機率 $\pi(s,a)$作為選擇動作的策略依據。而且在輸和贏的情況下有不同的學習率，分別為δ_l和δ_w。透過圖 3 中第 14 行的公式可判斷出此時要用哪一個學習率去更新策略 $\pi(s,a)$。透過設計兩種不同的學習率，能讓代理人更快使用並適時調整策略進而達到最佳結果。

01 設 α 與 $\delta_l > \delta_w$ 為學習率，並初始化以下參數

02 $Q(s,a) \leftarrow 0$ 、 $\pi(s,a) \leftarrow \frac{1}{|A_i|}$ 、 $\bar{\pi}(s,a) \leftarrow 0$ 、 $C(s) \leftarrow 0$

03 重複執行(在所有回合中):

04 初始化狀態 s

05 重複執行(在回合的每一步中):

06 (a)在狀態 s 以機率 $\pi(s,a)$ 選擇出動作 a，更新$Q(s,a)$

07 (b)執行動作 a，得到獎勵值 r 與下一狀態 s'

08 $$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a')\right)$$

09 (c)更新平均期望值$\bar{\pi}(s,a)$

10 $$C(s) \leftarrow C(s) + 1$$

11 $$\forall a' \in A_i \quad \bar{\pi}(s,a') \leftarrow \bar{\pi}(s,a') + \frac{1}{C(s)}\left(\pi(s,a') - \bar{\pi}(s,a')\right)$$

12 (d)更新 $\pi(s,a)$ 並正規化

13 $$\pi(s,a) \leftarrow \pi(s,a) + \begin{cases} \delta & \text{if } a = \arg\max_{a'} Q(s,a') \\ \frac{-\delta}{|A_i|-1} & \text{otherwise} \end{cases}$$

14 $$\delta = \begin{cases} \delta_w & \text{if } \Sigma_a \pi(s,a)Q(s,a) > \Sigma_a \bar{\pi}(s,a)Q(s,a) \\ \delta_l & \text{otherwise} \end{cases}$$

15 (e) 更新狀態 $s \leftarrow s'$

圖 3 WoLF-PHC 演算法

2.4 Cooperative Policy Mechanism

Cooperative Policy Mechanism(CPM) [3] 演算法是一種基於合作傾向值 K 和 Payoff Function 的新演算法。而合作傾向值是由 H. Shi 等人提出的[4]，它要求 Q-Table 記錄環境中所有代理人的狀態s和他們自己的行動a，這使得代理人能共享狀態和行動，形成一個完全溝通的群體代理人合作系統。在 CPM 演算法中，合作傾向值被重新定義為 $p_i(s, a)$，範圍在$(0, 1]$之間，以幫助代理人在無法觀察其他代理人行動的環境下學習。此外，PTable 和 QTable 都各自記錄代理人的狀態和行動。CPM 演算法使用的 Payoff Function 策略旨在最大化各自的收益並提升代理人間的合作能力，如圖 4 的第 7 至 10 行。此算法透過增強合作傾向，提高群體代理人對環境的探索速度和學習效率。圖 4 為完整的 CPM 演算法。

01 設 α 與 $\lambda_l > \lambda_w$ 為學習率、 w 為權重，並初始化以下參數

02 $Q_i(s,a) \leftarrow 0$ 、 $p_i(s,a) \leftarrow \frac{1}{|A_i|}$

03 重複執行(在所有回合中):

04 初始化狀態 s

05 重複執行(在回合的每一步中):

06 (a)計算 $Payoff(s,a)$

07 $Q_{\max}(s) \leftarrow \max_{a'} Q(s,a')$

08 $Q_{\min}(s) \leftarrow \min_{a'} Q(s,a')$

09 $Q_n(s,a) \leftarrow \frac{Q_i(s,a) - Q_{\min}(s)}{Q_{\max}(s) - Q_{\min}(s)}$

10 $Payoff(s,a) \leftarrow w * p_i(s,a) + (1-w) * Q_n(s,a)$

11 (b)選擇動作 $a \leftarrow \underset{a'}{argmax}\, Payoff(s,a')$

12 (c)執行動作 a，得到獎勵值 r 與下一狀態 s'

13 (d)觀察新狀態 s' ，並給定 K 值與 λ 值

14 $K \leftarrow \begin{cases} 0 & \text{當合作失敗} \\ 1 & \text{其他情況} \end{cases}$

15 $\lambda \leftarrow \begin{cases} \lambda_l & \text{當合作失敗} \\ \lambda_w & \text{其他情況} \end{cases}$

16 (e)更新 $p_i(s,a) \leftarrow p_i(s,a) + \lambda * (K - p_i(s,a))$

17 (f)更新 $Q_i(s,a) \leftarrow Q_i(s,a) + \alpha\left(r + \gamma \max_{a'} Q_i(s',a') - Q_i(s,a)\right)$

18 (g)更新狀態 $s \leftarrow s'$

圖 4 CPM 演算法

三、 實驗環境與結果

本章將介紹三種實驗環境並將三種演算法進行比較。第 3.1 節會先介紹 Ball Balance 的實驗環境，接著比較 Hysteretic Q-Learning 和 WoLF-PHC 兩個演算法的結果。第 3.2 節先介紹 Pursuit Domain 的實驗環境，接著比較三種演算法的結果。第 3.3 節會先介紹 Matching Pennies 的實驗環境，藉著比較 Hysteretic Q-Learning、WoLF-PHC 兩個演算法的結果。由於使用 CPM 時須能清楚分辨何謂勝負，因此在選定之實驗中只有 Pursuit Domain 能使用此演算法。

3.1 Ball Balance

3.1.1 實驗環境介紹

此實驗的目的是在一個由兩個機器人控制，位於兩端的平坦桌面上，保持一個滾動的球在中央平衡[5]，如圖 5 所示。對於每個機器人控制桌子的兩端進行分散式學習，完整的狀態由 x = $(x,\ \dot{x})$ 給出，動作是 a = $(h_1,\ h_2)$，機器人的動作分別為 $a_1 = h_1$和 $a_2 = h_2$，對於每個動作，使用在-1 和 1 之間均勻分佈的 15 個值。為了應用先前提出的強化學習形式，首先必須對時間軸以及連續狀態和動作進行離散化。樣本時間為 0.03 秒。對於狀態空間，選擇了 100×50 的離散化。因此，這產生了一個大小為 100×50×15 的兩個 Q_i - table 進行。而所有實驗都使用一個折扣因子 γ = 0.9，遵循 ε-greedy 動作選擇方法（ε = 0.1），並且繪製球體在機器人經過 5000 次連續訓練後的速度以及位置變化圖。

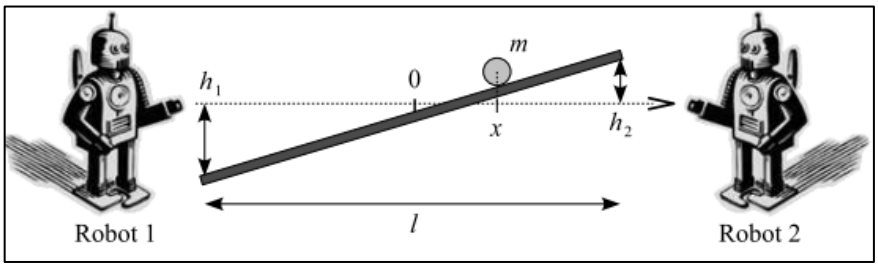


圖 5 平衡球任務示意圖

每次試驗都從初始狀態 x = (0.495, 1.041) 開始，最長進行 20 秒。如果球從桌子上掉下來，實驗就結束。球的動態特性為： m = 0.5、g = 9.8、l = 2、c = 0.01，目的在於算出加速度進而算出下一個狀態，也就是球的下一個位置和速度。其中有

限制速度介於-3 到 3 之間。以下(3)式為動態特性的公式。此實驗選定的獎勵函數如(4)式所示，因為較著重在球體的位置變化，因此獎勵函數中位置的係數設為 0.8，而速度的係數設為 0.2。

$$m\ddot{x} = -c\dot{x} + mg(\frac{h_1 - h_2}{l}) \tag{3}$$

$$r = 0.8e^{\frac{-x^2}{0.25^2}} + 0.2e^{\frac{-\dot{x}^2}{0.25^2}} \tag{4}$$

3.1.2 結果與討論

在圖 6 中顯示了球體的位置變化，其中橫軸代表回合數，縱軸代表球體的位置。而在圖 7 中則顯示了球體的速度變化，其中橫軸代表回合數，縱軸代表速度變化。由圖 6、圖 7 可看出，Hysteretic Q-Learning 的學習速率較 WoLF-PHC 快，同樣經過 20 回合 5000 次訓練，Hysteretic 能夠更快學習到如何達成目標並找出更快的解。相對的 WoLF-PHC 雖然也同樣能達成目標，但學習到的最佳解需晚 Hysteretic 大約 3 回合才能達到目標。因此我們認為在這個合作型任務中 Hysteretic Q-Learning 表現最佳。

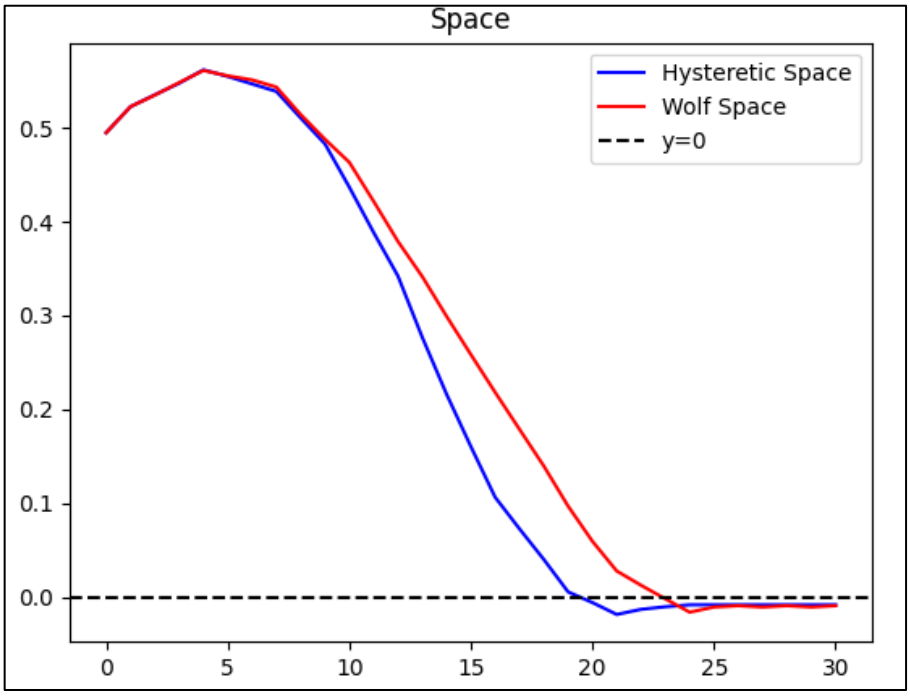


圖 6 球體位置比較圖

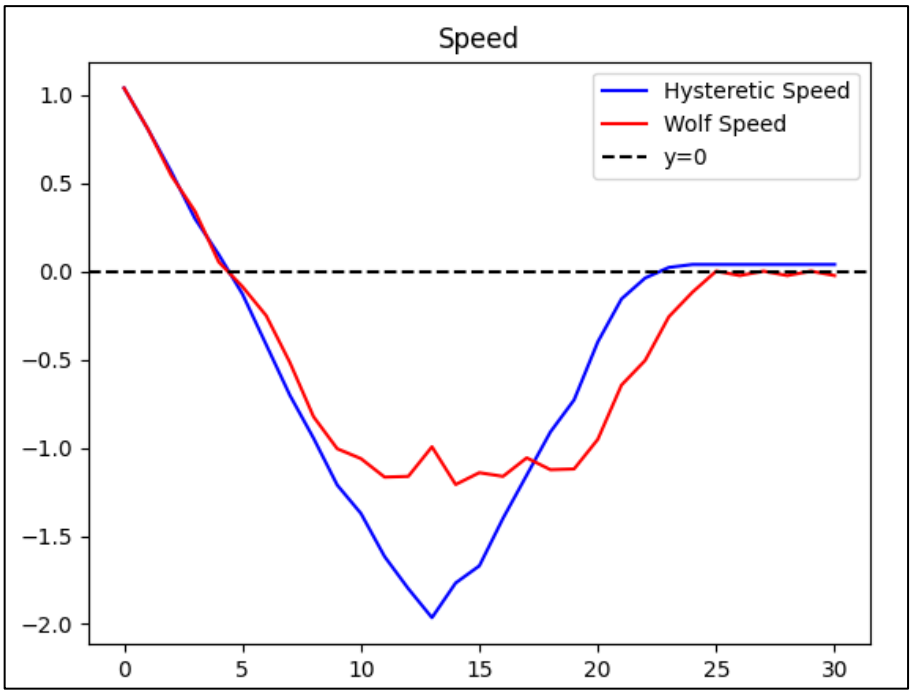


圖 7 球體速度比較圖

3.2 Pursuit Domain

3.2.1 實驗環境介紹

Pursuit Domain 是一個流行的多代理人合作的實驗，其中一些代理人被稱為捕食者，必須捕捉另一個代理人，被稱為獵物。實驗中使用一個離散的 10×10 大小的網格環境，其中兩個捕食者必須明確協調他們的行動，以捕捉一個獨立的獵物。在每一步，所有捕食者同時執行 5 種可能的行動之一：向北移動、向南移動、向東移動、向西移動或保持靜止。獵物則根據一個隨機策略移動：它以 0.2 的概率保持在當前位置，否則以均勻概率移動到其相鄰的空格之一。當兩個捕食者位於獵物相鄰的格子中，且其中一個捕食者移動到獵物的位置，而另一個捕食者則保持在其當前位置以提供支持時，獵物就被捕捉。狀態空間由兩個捕食者相對於獵物的相對位置表示。完整的狀態-動作空間則包括兩個捕食者相對位置的所有組合。總共產生了 100×100×100 種狀態的兩個捕食者的 Q- table。

當兩個捕食者最終位於同一格中時，它們會受到 R = -10 的懲罰，並被移至網格上的隨機空位置。捕獲時會給每個代理人 R = 37.5 的獎勵，隨後捕食者和獵物被放置到隨機位置。此外，當兩者中的一個捕食者沒有支援地移動到獵物時，每個捕

食者都會受到 R = -25 的懲罰，並且兩者都被移至隨機的空單元格。若捕食者撞牆會給撞牆之捕食者 R = -25.5 的懲罰，若未發生任何事情(上述提到的狀況)，則會給捕食者 R = -0.01 的懲罰，提醒他們採取的動作尚未達成目標。圖 8 是捕食者和獵物在網格環境的示意圖，紅圓點為兩個捕食者(agent1 和 agent2)，藍圓點為獵物(prey)。

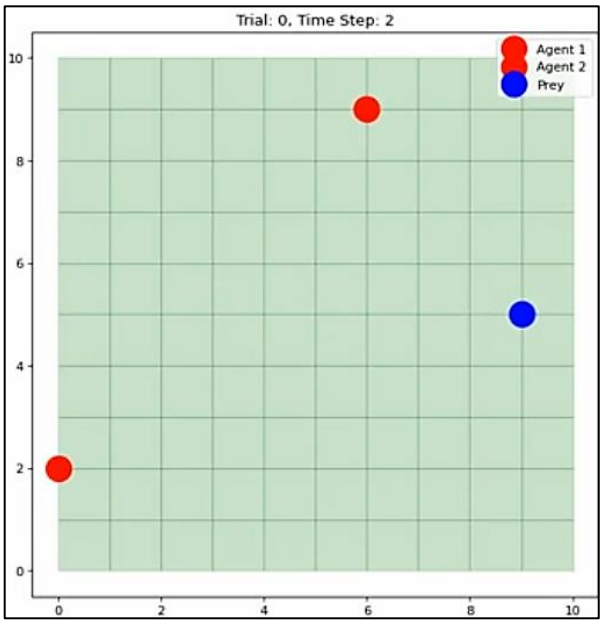


圖 8 網格環境示意圖

3.2.2 結果與討論

在這個實驗中，主要以成功捕捉一次所需最少步數和每回合平均捕捉成功次數這兩個結果來比較。由於實驗結果出乎於我們的意料，因此我們做了比較多種嘗試，並比較了在不同組參數以及條件下對於此實驗的影響。

首先在 Hysteretic Q-Learning 中測試了兩種不同的參數，分別是 α = 0.1、β = 0.01、γ = 0.9 (Paper) 和 α = 0.85、β = 0.55、γ = 0.9 (ES)。圖 9 中顯示出捕食者成功捕捉一次所需的最少步數，其中橫軸表示訓練次數，縱軸代表捕食者的步數。從圖 9 可看出在 Paper 這組參數下捕食者的步數大幅減少到 200 步以下。而圖 10 顯示了每回合平均捕捉成功次數，其中橫軸表示訓練次數，縱軸表示捕捉次數。從圖 10 可以看出捕捉次數在 Paper 這組參數下比在 ES 參數下還高。因此由圖 9、10 可討論出 Paper 的參數使得 agents 能更快學習好達成目標的方法，相對地雖然 ES 的

參數也能逐漸減少所需步數，但學習的速度卻沒有比較快。最後在比較三種演算法時會使用 Paper 這組參數的結果去討論。

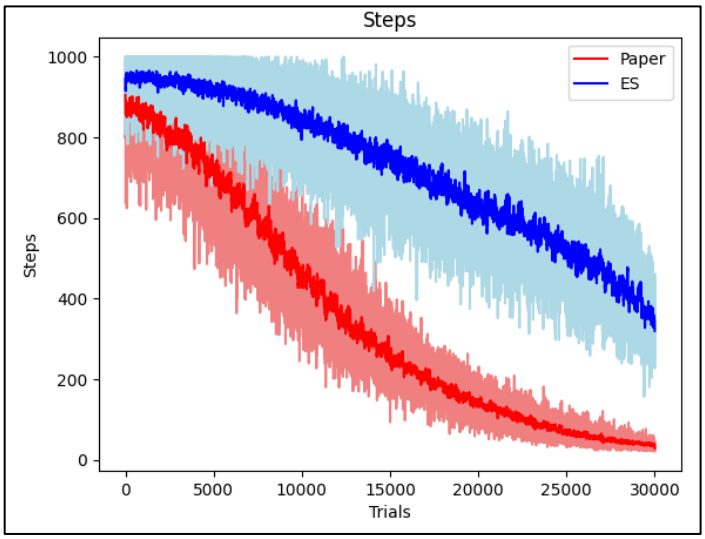


圖 9 成功捕捉一次所需最少步數-Hysteretic

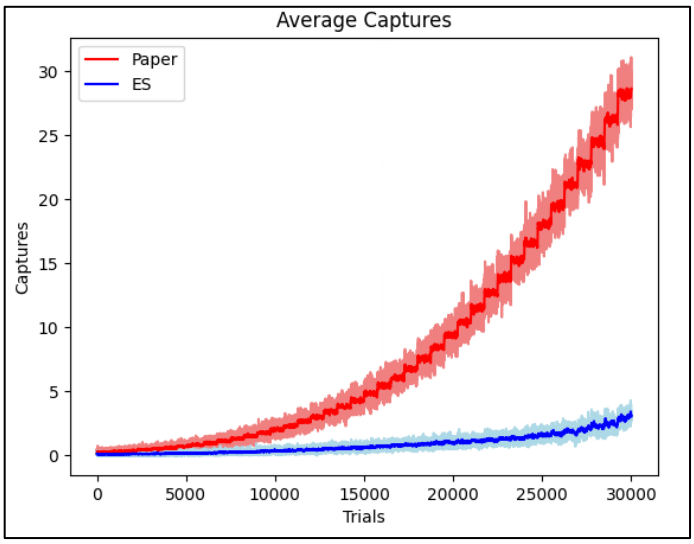


圖 10 每回合平均捕捉成功次數-Hysteretic

然而在 WoLF-PHC 演算法中，我們始終無法讓其學習曲線成功收斂在好的結果。因此我們試著使用兩種不同的狀態表示法，第一種是 agent 只能觀察到自己和獵物的位置，即 states = (agent, prey) ；第二種則是 agent 能觀察到自己、另一個 agent 和獵物的位置，即 states = (agent1, agent2, prey)。透過更改狀態的方式去觀察結果是否能成功收斂。圖 11 顯示了捕食者成功捕捉一次所需的最少步數，其中橫軸表示訓練次數，縱軸代表捕食者的步數。從圖 11 可以看出雖然使用第二種狀態表示法

能減少步數，但減少幅度不大且也無法收斂。而圖 12 顯示了每回合平均捕捉成功次數，其中橫軸表示訓練次數，縱軸表示捕捉次數。從圖 12 中也可看出不管是哪一種狀態表示法，對於捕捉成功次數都沒有太大的增加。因此在最後比較中會使用第二種狀態表示法的數據去討論。

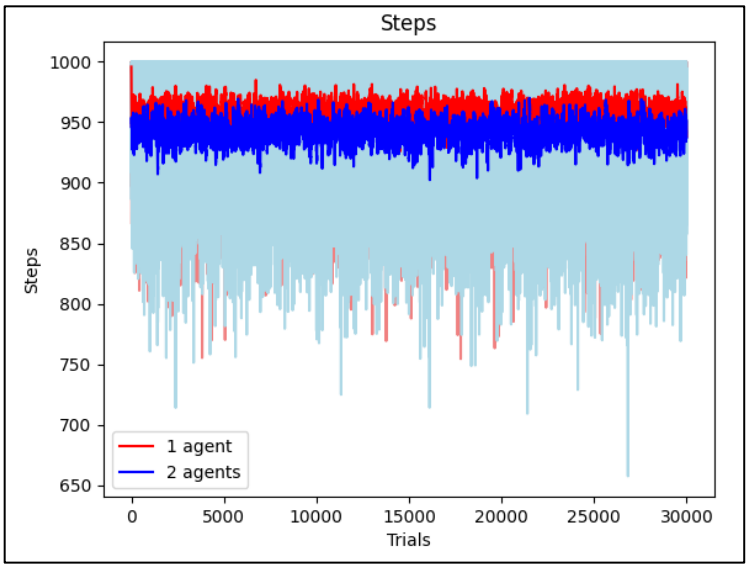


圖 11 成功捕捉一次所需最少步數-WoLF

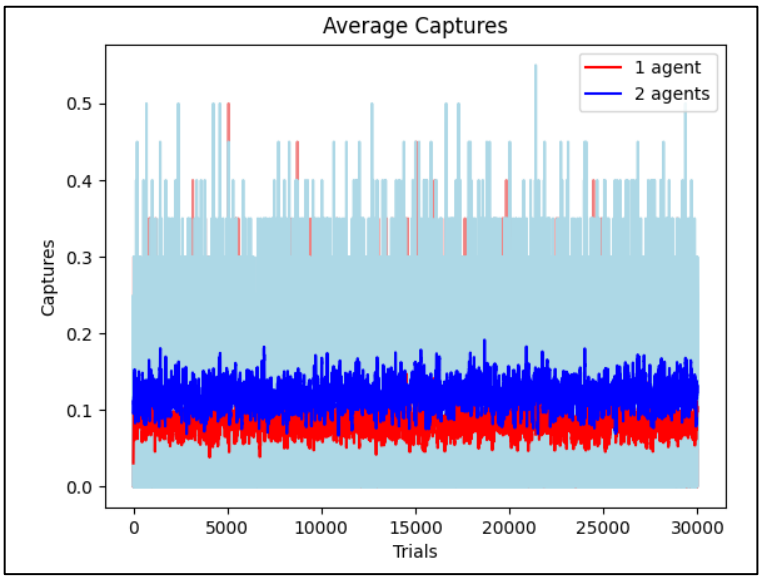


圖 12 每回合平均捕捉成功次數-WoLF

在 CPM 演算法中，由於我們一直無法找到最佳解，因此我們嘗試了最多種組合。包含了 ε 固定與不固定、λ 相同與不相同和兩組不同參數，來探討這三種參數對 CPM 的影響。

首先我們使用了兩種不同參數，分別是 $\alpha = 0.6$、$\gamma = 0.9$、$w = 0.3$、$\lambda_l = 0.3$、$\lambda_w = 0.6$ (Paper)和 $\alpha = 0.1$、$\gamma = 0.9$、$w = 0.3$、$\lambda_l = 0.3$、$\lambda_w = 0.6$(ES)。圖 13 中顯示出捕食者成功捕捉一次所需的最少步數，其中橫軸表示訓練次數，縱軸代表捕食者的步數。從圖 13 可看出在 ES 這組參數下步數較少且下降幅度較大。圖 14 顯示每回合平均捕捉成功次數。其中橫軸表示訓練次數，縱軸表示捕捉次數。由圖 14 也可看出在 ES 這組參數下表現較佳且 CPM 也能夠成功收斂到當前最佳解。因此後續的實驗則會繼續使用 ES 這組參數。

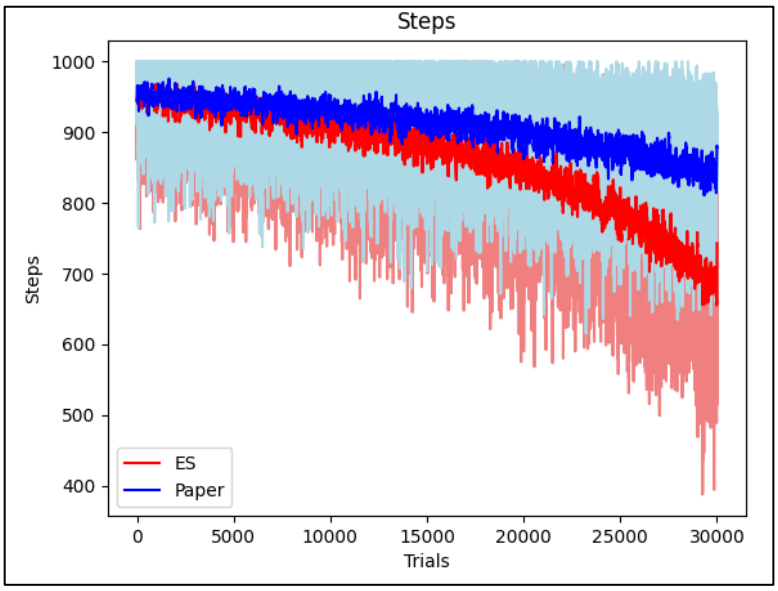


圖 13 成功捕捉一次所需的最少步數-CPM

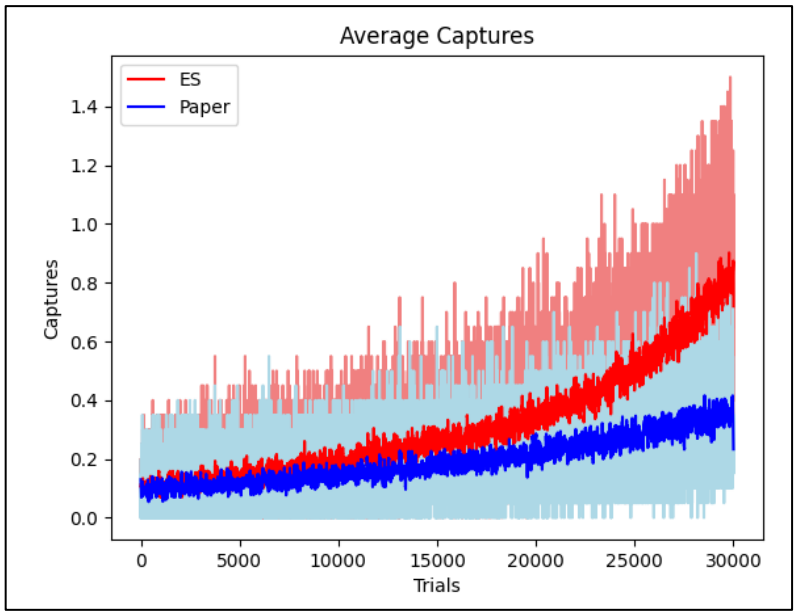


圖 14 每回合平均捕捉成功次數-CPM

接著我們開始針對 ε 進行討論。將 ε 參數分成固定和不固定。圖 15 顯示出捕食者成功捕捉一次所需的最少步數，其中橫軸表示訓練次數，縱軸代表捕食者的步數。圖 16 顯示了每回合平均捕捉成功次數。其中橫軸表示訓練次數，縱軸表示捕捉次數。由圖 15、圖 16 可以得知，當 ε 固定時會導致學習曲線沒辦法在好的結果上進行收斂。因此後續的實驗則繼續使用不固定的 ε 參數。

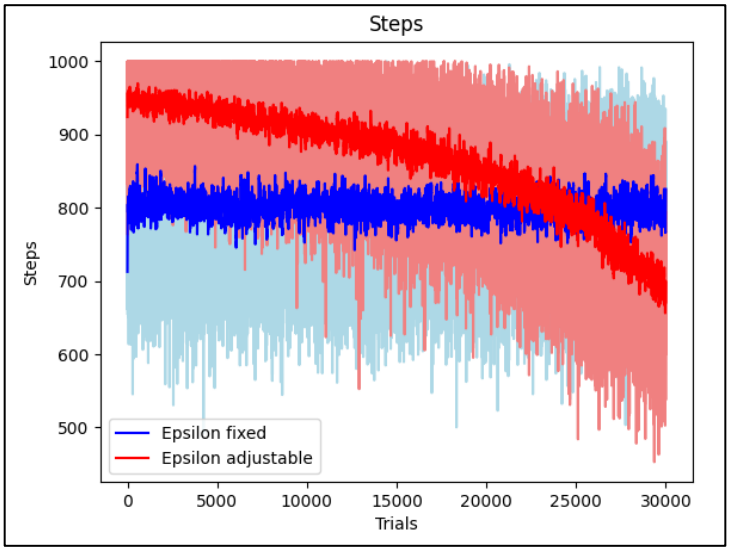


圖 15 成功捕捉一次所需最少步數-CPM(ε)

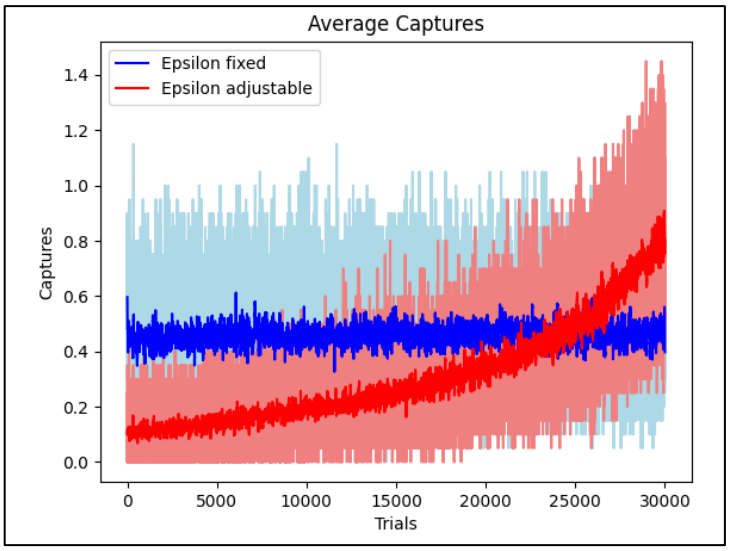


圖 16 每回合平均捕捉成功次數-CPM(ε)

最後我們將針對 λ 進行討論，將 λ 分成相同和不相同。圖 17 顯示出捕食者成功捕捉一次所需的最少步數，其中橫軸表示訓練次數，縱軸代表捕食者的步數。圖 18 顯示了每回合平均捕捉成功次數。其中橫軸表示訓練次數，縱軸表示捕捉次數。由圖 17、圖 18 可以得知，當 λ 相同時學習曲線的收斂速度會大幅減緩。因此在最後比較時會使用在不相同的 λ 參數下所產生的數據。

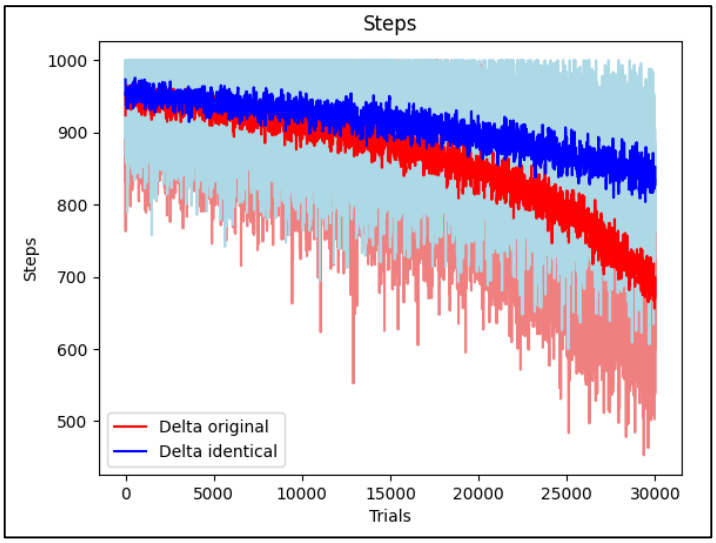


圖 17 成功捕捉一次所需最少步數-CPM(λ)

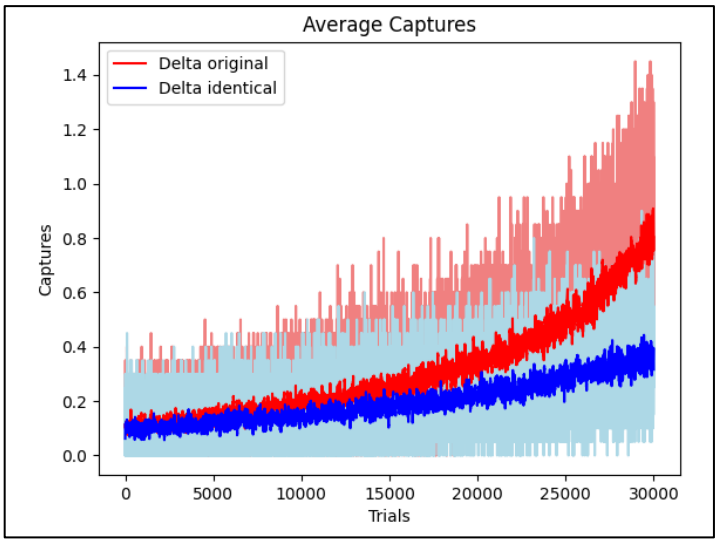


圖 18 每回合平均捕捉成功次數-CPM(λ)

最後我們將三種演算法所能產生的最佳學習曲線進行比較。由圖 19、圖 20 我們可以發現 Hysteretic Q-Learning 的學習速度遠超過 CPM 和 WoLF-PHC 所能學習的速度且表現十分良好，相對於 Hysteretic Q-Learning，CPM 和 WoLF-PHC 的平均捕捉次數均未能超過 5 次，可以得知此兩演算法十分不適於這個實驗。雖然 CPM 能夠緩慢趨向較好的結果，不過按其收斂速度可能仍須將回合數增加制 100000 才能達到如 Hysteretic Q-Learning 相同的結果。而 WoLF-PHC 則是幾乎完全沒有學習到的跡象，隨著回合數的增加也未產生進步的現象。即使將回合數延長至 100000 也

大機率無法達到更好的結果。因此我們認為在此合作型任務中 Hysteretic Q-Learning 演算法表現最佳。

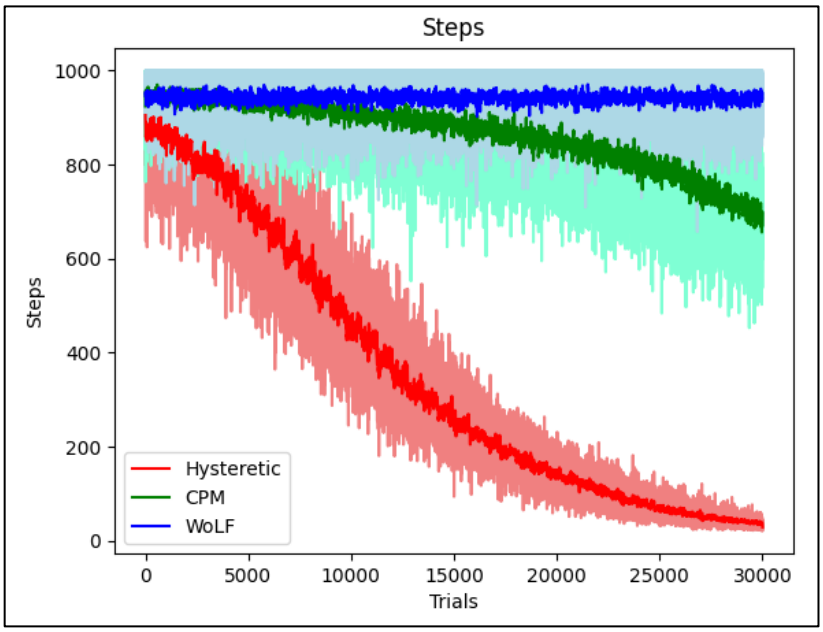


圖 19 成功捕捉一次所需最少步數

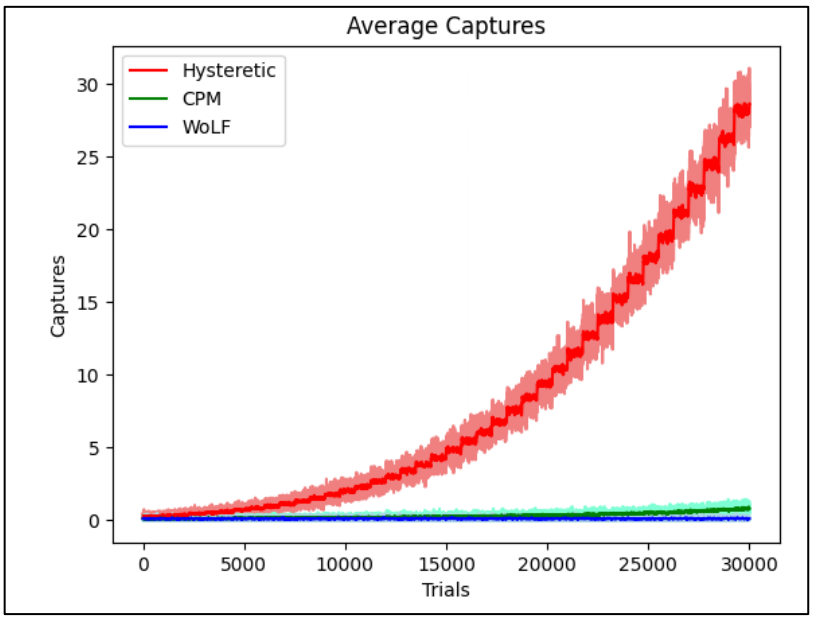


圖 20 每回合平均捕捉成功次數

3.3 Matching Pennies

3.3.1 實驗環境介紹

此實驗的定義是兩位代理人各有一枚硬幣，雙方可以決定自己要翻到哪一面，接著雙方同時出示自己翻的那一面，如果兩個人的硬幣是同一面，則一方贏錢，若不同面，則另一方獲勝。我們設定動作為擲出正面和擲出反面兩種。在此實驗中，我們輸出了其中一名代理人在遊戲中投出正面的比例，由於如果總是傾向於擲出正面，那麼對手就有動機選擇反面以獲勝，反之亦然。因此當每位玩家都以 1/2 的機率選擇正面和反面時才能避免被對手利用。此實驗選定的獎勵函數如下圖：

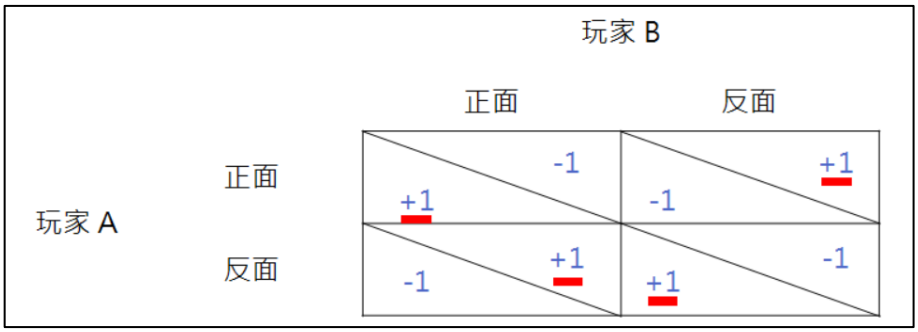


圖 21 獎勵函數圖

3.3.2 結果與討論

圖 22、圖 23 分別為代理人使用 Hysteretic Q-Learning 和 WoLF-PHC 所擲出正面比例的數據。其中橫軸為訓練次數，縱軸為擲出正面的比例。由圖 22 可以看出學習曲線的收斂速度較慢且不穩定。從圖 23 可看出學習曲線很快就穩定收斂到 0.5 的位置。

WoLF-PHC 在此實驗中能夠穩定並快速收斂在 0.5，相對地 Hysteretic Q-Learning 花費的時間不但是 WoLF-PHC 的兩倍且收斂地相當不穩定。因此在此競爭型任務中，我們認為 WoLF-PHC 演算法表現最佳。

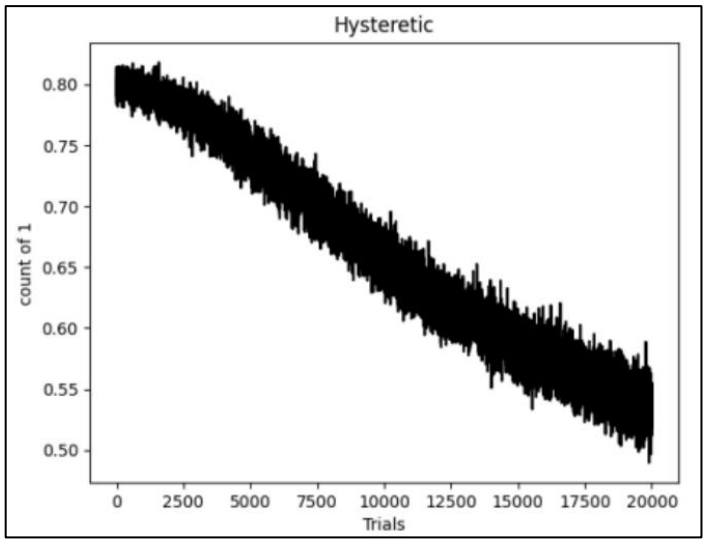


圖 22 擲出正面比例圖-Hysteretic

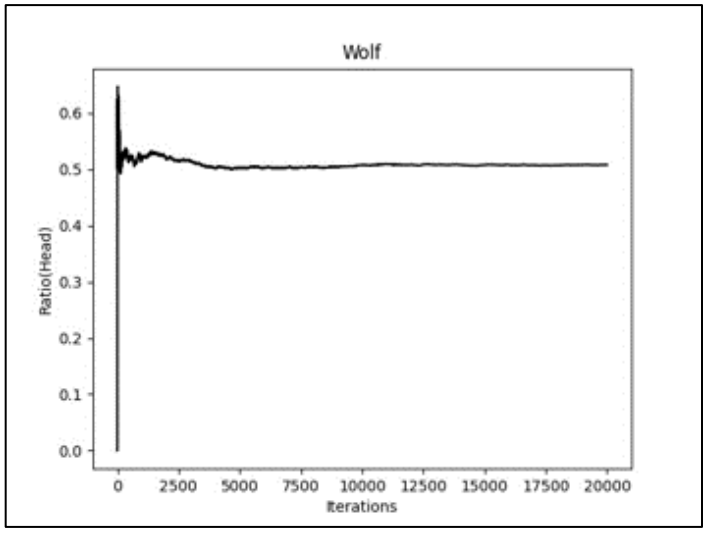


圖 23 擲出正面比例圖-WoLF-PHC

四、 結論

透過第三章所討論的實驗結果可歸納出在本專題選擇的合作型遊戲中 Hysteretic Q-Learning 演算法的結果都是最佳的。而在競爭遊戲上，則是 WoLF-PHC 演算法表現較佳。

CPM 是基於 Q-learning 演算法，理想上將兩個達成目標的條件（合作成功和達成目標）拆開並同時進行，收斂速度應該要比較快。然而根據實驗結果卻發現雖然比 WoLF-PHC 快但和 Hysteretic Q-Learning 仍相差甚遠。由此可知，合作成功跟達成目標對增強式學習而言，可以整合在同一個 Reward Function 中進行學習，分開成 reward 和合作驅勢並無明顯優勢。

參考文獻

[1] L. Matignon, G. J. Laurent, and N. L. Fort-Piat, “Hysteretic Q-Learning : an Algorithm for Decentralized Reinforcement Learning in Cooperative Multi-Agent Teams,” in *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems* (IROS), pp. 64-69, Nov. 2007.

[2] M. Bowling and M. Veloso, “Rational and Convergent Learning in Stochastic Games,” in *Proc. of the 17th International Joint Conference on Artificial Intelligence*, vol. 2, pp. 1021-1026, Aug. 2001.

[3] Y. T. Wang, “Cooperative Policy Mechanism using Reinforcement Learning for Multi-robot System”, Master Thesis, Dept. Elect. Eng., Tunghai Univ., Taiwan, 2023.

[4] H. Shi, L. Zhai, H Wu, Maxwell Hwang, K. S. Hwang, and H. P. Hsu, “A Multitier Reinforcement Learning Model for a Cooperative Multiagent System,” *IEEE Transactions on Cognitive and Developmental Systems*, vol. 12, no. 3, pp. 636-644, 2020.

[5] Collaborative-Ball-Balancing,
Available at: https://github.com/parqz/Collaborative-Ball-Balancing

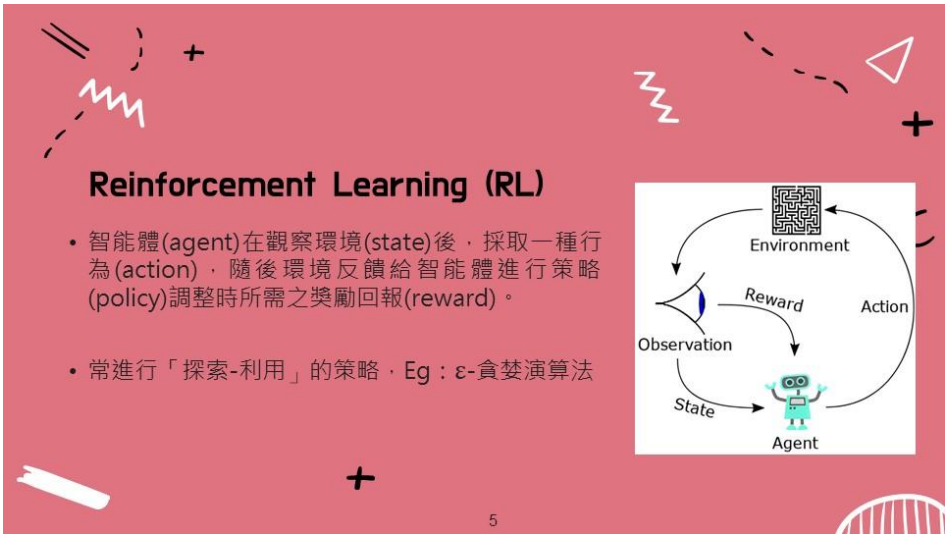
附錄 1 畢業專題簡報

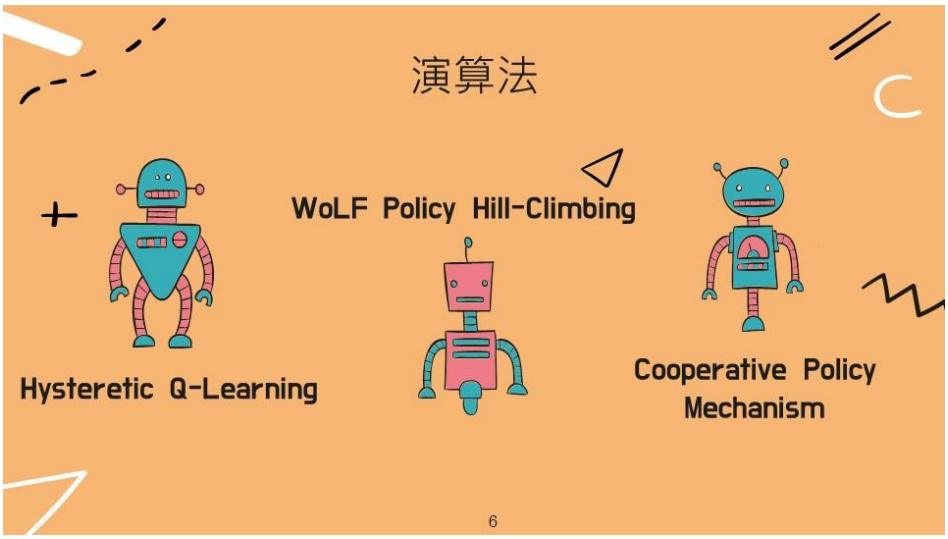
國立臺南大學資訊工程學系113級畢業專題
增強式學習演算法於多智能體系統中的競爭與合作策略探究
Exploration of Competitive and Cooperative Strategies of Reinforcement Learning Algorithms in Multi-Agent Systems
組別：第九組
組員：林奕君、陳馨柔
1

摘要
我們實作了Value-based的Hysteretic Q-Learning和CPM(Cooperative Policy Mechanism)，與Policy-based的WoLF PHC(Win or Learn Fast Policy Hill Climbing)共三種演算法，進行一種競爭型任務：Matching Pennies 和兩種合作型任務：Ball Balance 和Pursuit Domain 的實驗，並討論導致其結果的可能原因。
2

01
演算法
介紹在本專題中所使用到的演算法
02
實驗說明
介紹在本專題中所應用到的遊戲實驗
03
結論
根據實驗結果做出結論
3

01.
演算法
介紹在本專題中所使用到的演算法
4


Reinforcement Learning (RL)
• 智能體(agent)在觀察環境(state)後，採取一種行為(action)，隨後環境反饋給智能體進行策略(policy)調整時所需之獎勵回報(reward)。
• 常進行「探索-利用」的策略，Eg：ε-貪婪演算法
Environment
Reward
Action
Observation
State
Agent
5


演算法
WoLF Policy Hill-Climbing
Hysteretic Q-Learning
Cooperative Policy Mechanism
6

Hysteretic Q-Learning

- 不需要機器人之間進行額外的通信的Q-Learning算法
- 用於合作式MAS中協調學習的去中心化強化學習算法

設學習率$\alpha > \beta$，初始化 $Q_i(s,a_i) \leftarrow 0$

重複執行(在所有回合中):

  初始化狀態 s

  重複執行(在回合的每一步中):

    (a)在狀態 s 以 Q 決策選擇出動作 a_i

    (b)執行動作 a_i，得到獎勵值 r 與下一狀態 s'

    (c)更新$Q_i(s,a_i)$

$$\delta \leftarrow r + \gamma \max_{a'} Q_i(s',a') - Q_i(s,a_i)$$

$$Q_i(s,a_i) \leftarrow \begin{cases} Q_i(s,a_i) + \alpha\delta & \text{if } \delta \geq 0 \\ Q_i(s,a_i) + \beta\delta & \text{else} \end{cases}$$

    (d)更新狀態 $s \leftarrow s'$

7

WoLF Policy Hill-Climbing (WoLF-PHC)

- 以鼓勵收斂的方式改變算法使用的學習率
- 想法：輸的時候學得快，贏的時候學得慢
- 玩家們互相適應對方的策略變化

設 α 與 $\delta_l > \delta_w$ 為學習率，並初始化以下參數

$Q(s,a) \leftarrow 0$ 、 $\pi(s,a) \leftarrow \frac{1}{|A_i|}$ 、 $\bar{\pi}(s,a) \leftarrow 0$ 、 $C(s) \leftarrow 0$

重複執行(在所有回合中):

  初始化狀態 s

  重複執行(在回合的每一步中):

    (a)在狀態 s 以機率 $\pi(s,a)$ 選擇出動作 a，更新$Q(s,a)$

    (b)執行動作 a，得到獎勵值 r 與下一狀態 s'

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a')\right)$$

    (c)更新平均期望值$\bar{\pi}(s,a)$

$$C(s) \leftarrow C(s) + 1$$

$$\forall a' \in A_i \quad \bar{\pi}(s,a') \leftarrow \bar{\pi}(s,a') + \frac{1}{C(s)}\left(\pi(s,a') - \bar{\pi}(s,a')\right)$$

    (d)更新 $\pi(s,a)$ 並正規化

$$\pi(s,a) \leftarrow \pi(s,a) + \begin{cases} \delta & \text{if } a = \arg\max_{a'} Q(s,a') \\ \frac{-\delta}{|A_i|-1} & \text{otherwise} \end{cases}$$

$$\delta = \begin{cases} \delta_w & \text{if } \Sigma_a \pi(s,a)Q(s,a) > \Sigma_a \bar{\pi}(s,a)Q(s,a) \\ \delta_l & \text{otherwise} \end{cases}$$

    (e) 更新狀態 $s \leftarrow s'$

8

Cooperative Policy Mechanism (CPM)

- 利用合作傾向值 $M(a^{-})$ 與 Payoff function 的概念所設計的新演算法
- 分成Q-Table和P-Table分別記錄代理人的獎勵值以及合作傾向值

設 α 與 $\lambda_l > \lambda_w$為學習率， w 為權重，並初始化以下參數

$Q_i(s,a) \leftarrow 0$ 、 $p_i(s,a) \leftarrow \frac{1}{|A_i|}$

重複執行(在所有回合中):

  初始化狀態 s

  重複執行(在回合的每一步中):

    (a)計算 $Payoff(s,a)$

$$Q_{\max}(s) \leftarrow \max_{a'} Q(s,a')$$

$$Q_{\min}(s) \leftarrow \min_{a'} Q(s,a')$$

$$Q_n(s,a) \leftarrow \frac{Q_i(s,a) - Q_{\min}(s)}{Q_{\max}(s) - Q_{\min}(s)}$$

$$Payoff(s,a) \leftarrow w * p_i(s,a) + (1-w) * Q_n(s,a)$$

    (b)選擇動作 $a \leftarrow \arg\max_{a'} Payoff(s,a')$

    (c)執行動作 a，得到獎勵值 r 與下一狀態 s'

    (d)觀察新狀態 s' ，並給定 K值與 λ值

$$K \leftarrow \begin{cases} 0 & \text{當合作失敗} \\ 1 & \text{其他情況} \end{cases}$$

$$\lambda \leftarrow \begin{cases} \lambda_l & \text{當合作失敗} \\ \lambda_w & \text{其他情況} \end{cases}$$

    (e)更新 $p_i(s,a) \leftarrow p_i(s,a) * \lambda * (K - p_i(s,a))$

    (f)更新 $Q_i(s,a) \leftarrow Q_i(s,a) + \alpha\left(r + \gamma \max_{a'} Q_i(s',a') - Q_i(s,a)\right)$

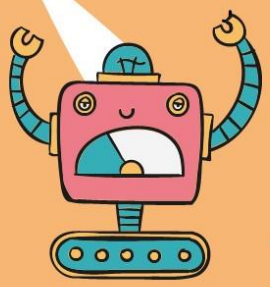
    (g)更新狀態 $s \leftarrow s'$

9

02.

實驗

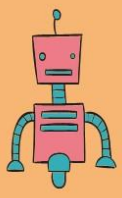
介紹在本專題中所應用到的遊戲實驗



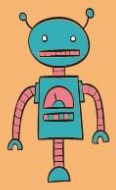
10

實驗說明

Matching Pennies



Ball Balance



Pursuit Domain

11

Matching Pennies

- 兩位agent各有一枚硬幣，雙方可以決定要翻到哪一面，接著雙方同時出示自己翻的那一面，若兩個人的硬幣是同一面，則一方獲勝並得到獎勵，若不同面則另一方獲勝並得到獎勵。
- 狀態(state)：state = (action1, action2)
- 動作(action)：擲出正面、擲出反面
- 結果:當每位玩家都以1/2的機率選擇正面和反面時才能避免被對手利用

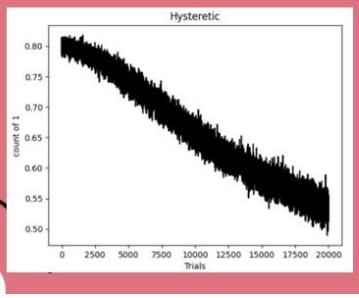
獎勵函數:

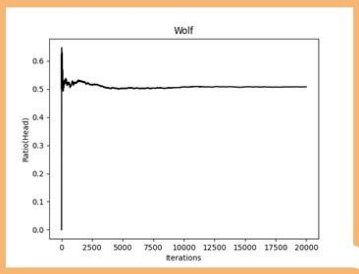
		玩家 B	
		正面	反面
玩家 A	正面	-1 / +1	+1 / -1
	反面	+1 / -1	-1 / +1

12

Matching Pennies

- WoLF-PHC能夠穩定並快速收斂在1/2的機率
- Hysteretic花費的時間不但是WoLF-PHC的兩倍且收斂地相當不穩定



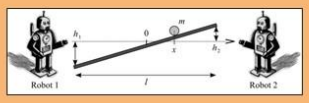


13

Ball Balance

- 此實驗的目的是在一個由兩個機器人控制、位於兩端的平坦桌面上，保持一個滾動的球在中央平衡
- 狀態(state)：state = $(x, \dot{x})$
- 動作(action)：a = (h_1, h_2)。對於每個 h_1、h_2，使用在-1和1之間均勻分佈的15個值
- 物理參數：m = 0.5、g = 9.8、l = 2、c = 0.01

示意圖:



動態特性:

$$m\ddot{x} = -c\dot{x} + mg\left(\frac{h_1 - h_2}{l}\right)$$

獎勵函數:

$$r = 0.8e^{-\frac{x^2}{0.25^2}} + 0.2e^{-\frac{\dot{x}^2}{0.25^2}}.$$

14

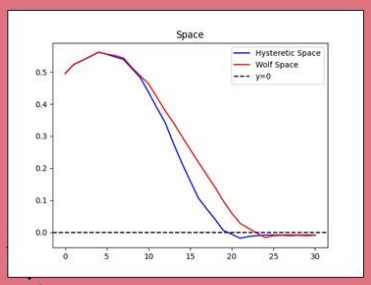
Ball Balance展示影片

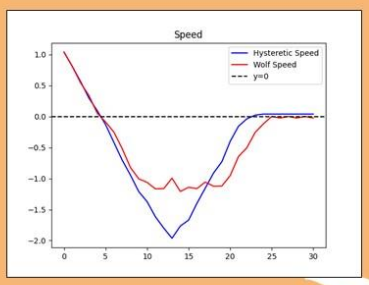
[redacted]

15

Ball Balance

- Hysteretic Q-Learning的學習速率較WoLF-PHC快
- WoLF-PHC學習到的最佳解需晚Hysteretic大約3回合才能達到目標



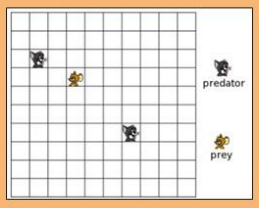


16

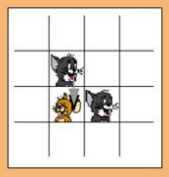
Pursuit Domain

- 使用一個離散的10×10環形網格環境，兩捕食者必須明確協調他們的行動以捕捉一個獨立的獵物
- 狀態(state)：state = (agent1, agent2, prey)
- 動作(action)：向北移動、向南移動、向東移動、向西移動或保持靜止

實驗環境示意圖:

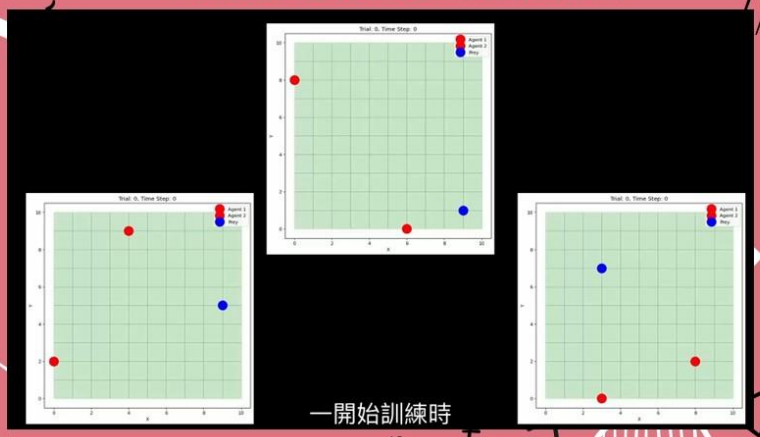


捕捉成功示意圖:



17

Pursuit Domain展示影片

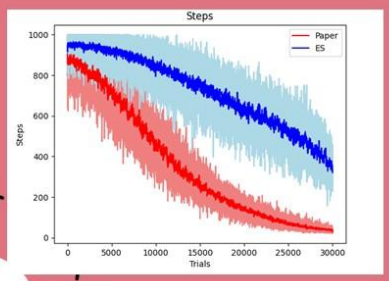


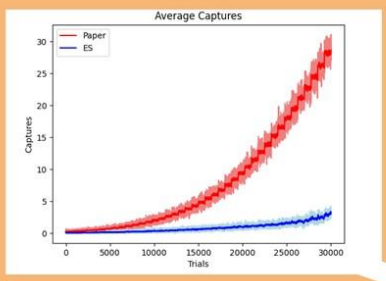
一開始訓練時

18

Pursuit Domain - Hysteretic Q-Learning

- α= 0.1、β= 0.01、γ= 0.9 (Paper)
- α= 0.85、β= 0.55、γ= 0.9 (ES)
- Paper的參數使得agents能更快學習好達成目標的方法

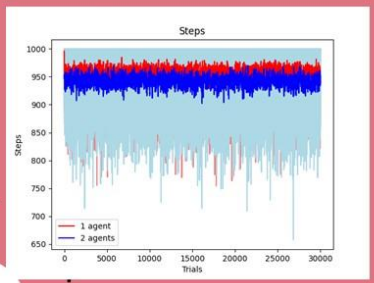


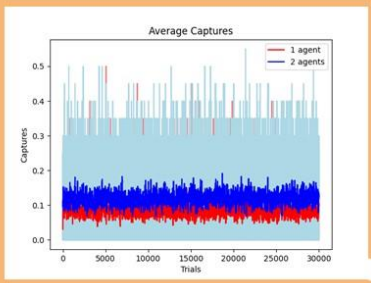


19

Pursuit Domain - WoLF-PHC

- agent只能觀察到自己和獵物的位置，即states = (agent, prey)
- agent能觀察到自己、另一個agent和獵物的位置，即states = (agent1, agent2, prey)

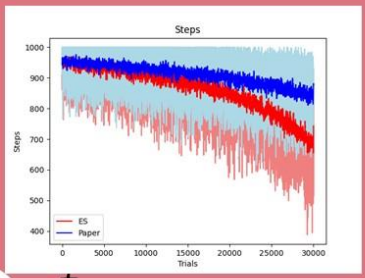


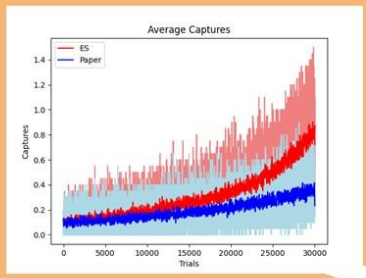


20

Pursuit Domain - CPM

- α= 0.6、γ= 0.9、w = 0.3、λ_l = 0.3、λ_w = 0.6(Paper)
- α= 0.1、γ= 0.9、w = 0.3、λ_l = 0.3、λ_w = 0.6(ES)
- ES這組參數的結果較佳

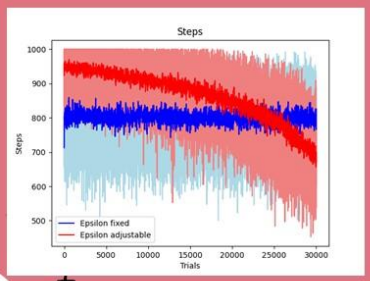


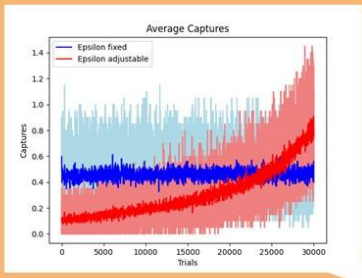


21

Pursuit Domain - CPM

- 當ε固定時會導致學習曲線沒辦法在好的結果上進行收斂

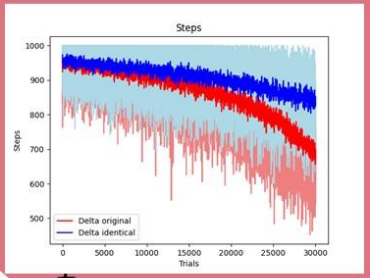


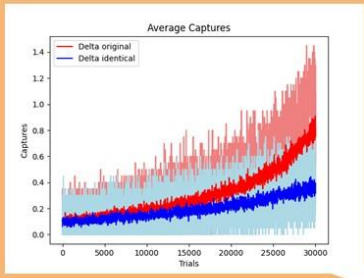


22

Pursuit Domain - CPM

- 當λ固定時學習曲線的收斂速度會大幅減緩

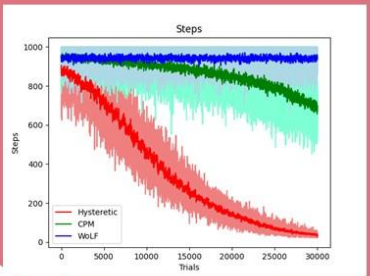


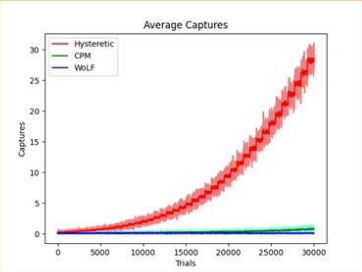


23

Pursuit Domain

- Hysteretic的學習速度遠超過CPM和WoLF-PHC
- CPM能夠緩慢趨向較好的結果，但仍需增加回合數
- WoLF-PHC則是幾乎完全沒有學習到的跡象





24

03.
結論
根據實驗結果做出結論
25

結論
(1) 透過實驗結果可歸納出在本專題選擇的合作型遊戲中Hysteretic Q-Learning演算法的結果都是最佳的。在競爭遊戲上，則是WoLF-PHC演算法表現較佳。
(2) CPM是基於Q-learning演算法，理想上將兩個達成目標的條件（合作成功和達成目標）拆開並同時進行，收斂速度應該要比較快。然而根據實驗結果卻發現雖然比WoLF-PHC快但和Hysteretic Q-Learning仍相差甚遠。
26

Thank you
for listening
27